Home Control Application Manual

Overview

The Home Control application is a Python-based system for managing home automation devices. It provides a web interface to control lights and sprinklers, and includes an automated scheduler based on local sunrise/sunset times.

Features

- **Web Interface**: Control devices from any browser on the local network.
- **Device Groups**: Organized into Lights A (RF), Lights B (MQTT), and Sprinklers.
- **Automated Scheduling**: Schedule events relative to sunrise/sunset or at specific times.
- **Sprinkler Timer**: Configure sprinkler duration and days via the web UI.
- **Lights B Timers**: Per-light ON/OFF timers (each light is its own Tasmota device, so every timer lives on that light's own IP/topic).
- **Lights A Schedule**: Edits the RF (group 1) schedule stored in `config.json` and regenerates the `at` jobs.
- **Modern UI**: Dark mode interface with responsive design.

Installation

Prerequisites

- Python 3.x
- `pip` (Python package manager)
- `mosquitto` (MQTT broker) and `mosquitto-clients`
- `netcat` (nc)

Dependencies

Install the required Python packages:

```
pip install flask paho-mqtt suntime
```

Configuration (`config.json`)

The application is configured via `config.json`. This file contains three main sections:

1. MQTT

Configure your MQTT broker settings:

```
"mqtt": {
    "broker": "hcontrol.local",
    "port": 1883,
    "topic_cmnd": "cmnd/tasmota_sprinklers/",
    "topic_stat": "stat/tasmota_sprinklers/RESULT"
}
```

2. Devices

Define your devices. Each device has a unique ID (e.g., "a1") and properties: - `name`: Display name. - `group`: "1" (RF/Lights A), "2" (MQTT/Lights B), or "3" (Sprinklers). - `swno`: Switch number (for multi-channel devices).

3. Location & Schedule

Set your location for sun calculations and define automated events:

```
"location": {
    "latitude": 37.37006,
    "longitude": -122.08839
},
"schedule": [
    {"time": "sunset", "offset": "-00:30:00", "device": "a1", "action": "on"},
    {"time": "today",  "offset": "22:00:00",  "device": "a1", "action": "off"}
]
```

Usage

Starting the Server

Run the server using Python:

`python3 hc_server.py`

The web interface will be available at `http://localhost:80` (or your server's IP).

Web Interface

- **Navigation**: Use the top buttons to switch between device groups.
- **Control**: Click a device button to toggle it ON (Green) or OFF (Red).
- **Configuration**: In the "Sprinklers" group, click "Configuration" to set sprinkler timers (start time + duration + days, all on the sprinkler Tasmota device).
- **Lights A Configuration**: In the "Lights A" group, click "Configuration" to edit the RF schedule:
  - Every group-1 light is listed with its `config.json` `schedule` entries (a light may have 1-4 entries).
  - Each entry is `[On|Off] [Today|Tomorrow|Sunrise|Sunset] [+|-] [HH:MM:SS]`; tick `delete` to remove it, use the blank `add:` row to append one.
  - Hours/minutes shift the base time; the **seconds field is the RF stagger delay** (`sleep N` before the command), matching `update_crontab.py`.
  - "next" shows the computed run time for that entry. Save rewrites `config.json` atomically (backup `config.json.bak.<timestamp>`, owner/mode preserved), hot-reloads the server config (keeping button states), and runs `update_crontab.py` **as the ubuntu user** to regenerate the `at` queue.
  - Entries for devices that are not in group 1 are preserved untouched and are not shown.
- **Lights B Configuration**: In the "Lights B" group, click "Configuration" to edit each light's two timers:
  - `Timer1` turns the light ON, `Timer2` turns it OFF.
  - Each timer row has an ON/OFF enable box, a mode (`At time`, `Sunrise`, `Sunset`), a `+/-` offset sign and the time/offset value.
  - One shared Mon–Sun row applies to both timers; the per-light master box writes `Timers ON/OFF` on that device.
  - Unreachable lights are shown with last known values and are skipped when saving.
  - Timers are written with the Tasmota key `Enable` (older `Arm` is ignored by current firmware), and lights are read from `stat/tasmota_<device>/RESULT` plus `stat/tasmota_<device>/STATUS5` for the IP display (IPs cached in `light_ip_cache.json`).

Automated Scheduling

To update the system scheduler (using `at` jobs), run:

`python3 update_crontab.py`

Notes: - Only `at` jobs containing `echo rf` (the home-control signature) are removed and recreated; unrelated `at` jobs are left untouched. - An entry whose time has already passed today is scheduled for tomorrow instead of failing (`at` refuses past times). - The service runs as `ubuntu` and is granted only `CAP_NET_BIND_SERVICE` (port 80). This script reads the `schedule` from `config.json`, calculates the exact times for today/tomorrow, and schedules the commands. It is recommended to run this script automatically (e.g., via cron) every day after midnight.

Troubleshooting

- **Logs**: Check the console output for logs. Errors are logged with timestamps.

- **MQTT**: Ensure your MQTT broker is running and accessible.
- **Permissions**: If running on port 80, you may need sudo or capability configuration.